LABSHEET 9

Object Detection and Recognition using Deep Learning Models on Standard Image Datasets

Aim

To implement a deep learning-based object detection and recognition system using a pre-trained model and evaluate its performance on images by identifying and localizing multiple objects.

Theory

Object detection combines object localization and classification. A detector predicts bounding boxes around objects and assigns class labels with confidence scores. This lab uses a pre-trained YOLO model from the Ultralytics library. Since the model has already learned visual features from a large dataset, it can detect common object categories without training from scratch.

Procedure

1. Install the required Python libraries: Ultralytics, OpenCV, and Matplotlib.
2. Load a pre-trained YOLO model.
3. Provide an image path or use the model's supported sample image.
4. Run inference to obtain object labels, confidence scores, and bounding boxes.
5. Save and display the image with detected objects.
6. Review the detections and note false positives or missed objects.

Source Code

Install dependencies first: `pip install ultralytics opencv-python matplotlib`

```
"""
Labsheet 9: Object Detection using a pre-trained YOLO model
Usage:
    python labsheet9_object_detection.py path/to/image.jpg
If no image path is supplied, the script downloads/uses Ultralytics' sample image.
Results are saved in the runs/detect/predict folder by default.
"""
import sys
from pathlib import Path
import cv2
import matplotlib.pyplot as plt
from ultralytics import YOLO

# Load a pre-trained YOLO model. The weights are downloaded on first run.
model = YOLO("yolov8n.pt")  # nano model: small and relatively fast

# Use an image supplied on the command line, or a sample image URL.
if len(sys.argv) &gt; 1:
    image_source = sys.argv[1]
else:
    image_source = "https://ultralytics.com/images/bus.jpg"

# Run inference and save an annotated image.
```

```
results = model.predict(
    source=image_source,
    conf=0.25,       # minimum confidence threshold
    iou=0.45,        # IoU threshold used by non-maximum suppression
    save=True,
    verbose=False
)

# Print detected classes, confidence scores, and bounding-box coordinates.
for result in results:
    names = result.names
    boxes = result.boxes
    if boxes is None or len(boxes) == 0:
        print("No objects detected.")
        continue

    print(f"Image: {result.path}")
    for box in boxes:
        class_id = int(box.cls[0].item())
        confidence = float(box.conf[0].item())
        x1, y1, x2, y2 = [float(v) for v in box.xyxy[0].tolist()]
        print(
            f"Class: {names[class_id]}, "
            f"confidence: {confidence:.2f}, "
            f"box: ({x1:.1f}, {y1:.1f}, {x2:.1f}, {y2:.1f})"
        )

    # Display the annotated image in a window using Matplotlib.
    annotated_bgr = result.plot()
    annotated_rgb = cv2.cvtColor(annotated_bgr, cv2.COLOR_BGR2RGB)
    plt.figure(figsize=(10, 7))
    plt.imshow(annotated_rgb)
    plt.axis("off")
    plt.title("YOLO Object Detection")
    plt.tight_layout()
    plt.show()
```

Questions and Answers

1. Differentiate between image classification, object detection, and image segmentation.

Image classification assigns one or more labels to an entire image, such as “car.” Object detection identifies individual objects and returns a class label and bounding box for each one. Image segmentation assigns a class or instance label to pixels, giving a more precise outline of each object. In short: classification labels the image, detection labels and locates objects, and segmentation labels their pixels.

2. Explain the working principle of the YOLO (You Only Look Once) object detection algorithm.

YOLO processes an image in a single forward pass through a neural network. The network predicts candidate bounding boxes, objectness/confidence values, and class probabilities. These predictions are filtered using confidence thresholds and non-maximum suppression (NMS), which removes overlapping boxes for the same object. The remaining boxes provide each detected object's location, class, and confidence. Modern YOLO versions use multi-scale features to detect objects of different sizes.

3. Compare YOLO, SSD, and Faster R-CNN based on speed, accuracy, and practical applications.

YOLO is a one-stage detector that predicts boxes and classes directly, offering high speed and a strong accuracy-speed balance; it is suitable for real-time video, robotics, and surveillance. SSD is also a one-stage detector and is generally fast, including on resource-limited devices, though accuracy can be weaker for small objects depending on the model. Faster R-CNN is a two-stage detector: a region proposal network suggests regions and a second stage classifies/refines them. It can provide strong accuracy but is usually slower and more computationally demanding. Actual results depend on model size, hardware, dataset, and settings.

4. What is a bounding box? Why is it important in object detection?

A bounding box is a rectangle enclosing a detected object, commonly represented by coordinates (x1, y1, x2, y2) or by centre, width, and height. It specifies where the object is in the image, allows objects to be counted or cropped, supports tracking across frames, and enables localization accuracy to be measured using IoU.

5. Explain the significance of confidence score and Intersection over Union (IoU) in object detection.

A confidence score indicates how confident the model is that a prediction corresponds to an object and/or a particular class. A threshold removes weak predictions, trading off false positives and missed detections. Intersection over Union (IoU) measures overlap between a predicted box and a ground-truth box: IoU = area of intersection / area of union. IoU is used to assess localization quality and in non-maximum suppression to remove duplicate overlapping predictions. A higher IoU means a closer box match.

6. What are the advantages of using pre-trained deep learning models over traditional image processing techniques?

Pre-trained models learn complex visual features from large datasets and can recognize varied objects without hand-designed rules for each object. They are often more robust to changes in pose, scale, background, and lighting than simple thresholding or edge-based methods. They also reduce development time and can be adapted to a task through fine-tuning. However, they may need substantial compute, can inherit dataset biases, and do not guarantee correct detections in every scene.

7. Why are datasets such as MS COCO and Pascal VOC widely used for object detection research?

MS COCO and Pascal VOC provide labeled images with object categories and bounding-box annotations, enabling models to be trained and evaluated on common benchmarks. Their established train/validation/test splits and evaluation conventions make comparisons more reproducible. COCO contains many categories and diverse, often crowded scenes; Pascal VOC is a widely used earlier benchmark with a smaller set of common categories.

8. Mention five real-world applications of object detection in computer vision.

1. Autonomous driving: detecting vehicles, pedestrians, cyclists, and traffic signs. 2. Video surveillance: detecting people, vehicles, and suspicious events. 3. Healthcare: locating structures or findings in medical images as decision-support. 4. Retail: shelf monitoring, inventory counting, and checkout automation. 5. Industrial inspection: identifying defects, components, or safety hazards on production lines.

9. What challenges are commonly encountered while detecting objects in complex real-world environments?

Common challenges include partial occlusion, small or distant objects, crowded scenes, poor lighting, shadows, motion blur, unusual viewpoints, changes in scale, and visually similar classes. Background clutter can cause false positives, while limited or imbalanced training data can cause missed detections. Real-time systems must also balance accuracy with latency, memory, and processing power.

10. How can object detection systems be further improved using recent advancements in deep learning and computer vision?

Performance can be improved by using stronger architectures and feature pyramids, training with larger and more diverse datasets, data augmentation, and fine-tuning on domain-specific examples. Transformer-based detectors, improved attention mechanisms, and better multi-scale feature learning can help in difficult scenes. Model distillation, pruning, quantization, and efficient inference engines can reduce latency. Combining detection with tracking, segmentation, temporal information, or sensor fusion can improve system-level reliability. Evaluation should use suitable metrics such as precision, recall, mAP, latency, and performance on edge cases.

Observations / Expected Output

The program displays an image with bounding boxes and class labels for detected objects. It also prints each detected class, confidence score, and box coordinates. Exact detections and confidence values depend on the selected image, model weights, and software version; therefore, they should be recorded from the user's actual run rather than assumed in advance.

Result

A pre-trained YOLO object detection model was implemented to identify and localize objects in an image. The output includes bounding boxes, class labels, and confidence scores, demonstrating the basic deep learning object detection pipeline.

Conclusion

This lab demonstrates object detection using a pre-trained deep learning model. YOLO performs detection in a single forward pass and provides a practical balance of speed and accuracy. Detection quality depends on the training data, object size, scene conditions, confidence threshold, and available computing resources.